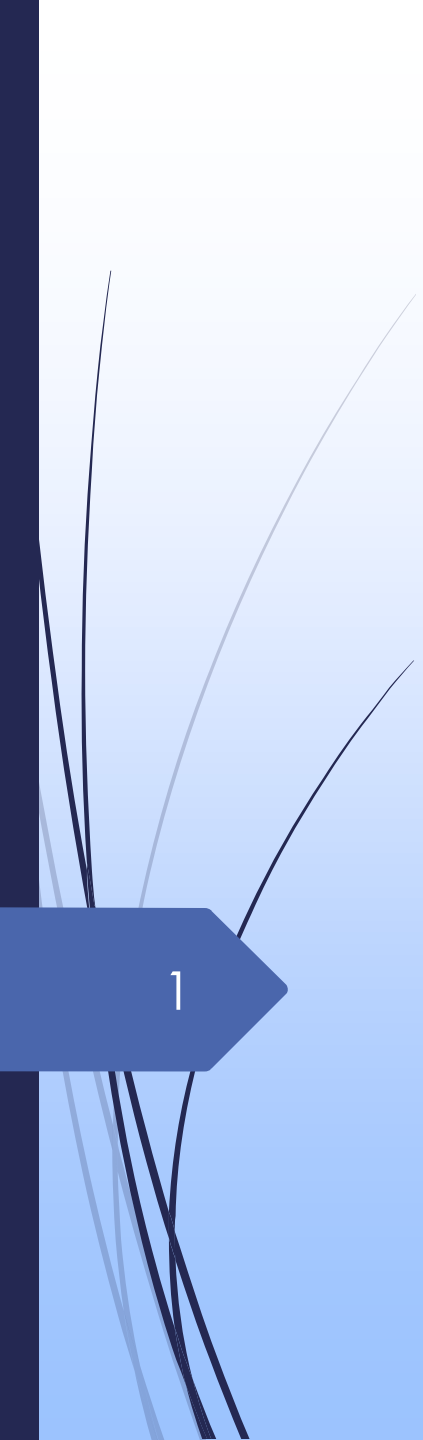




EECS 280 – Lecture 3

Procedural Abstraction

1

Procedural Abstraction

- In general...
 - ...helps manage complexity.
 - ...hides details.

Procedural Abstraction

- Example:
Making A PBJ sandwich



what = how

- what**
The abstract idea,
a holistic view.



what what = how

what

- How**
All the details.
Don't necessarily
need to know this.



what = how

what

what



what

what

what

High Level

Low Level

Procedural Abstraction

- In general...
 - ...helps manage complexity.
 - ...hides details.
- In computer programs...
 - ...makes programs easier to maintain and modify.
 - ...separates **what** code does from **how** it works.
Functions are the main tool for this in C++.

Interfaces vs. Implementations

- ▶ In CS terminology...
 - ▶ an **interface** specifies **what** something does.
 - ▶ an **implementation** specifies **how** it works.
- ▶ For example, consider an ATM machine.
 - ▶ Most ATMs have very similar **interfaces**:
 - ▶ Card reader
 - ▶ Screen for display
 - ▶ Buttons for withdraw/deposit
 - ▶ But internally may use different **implementations**:
 - ▶ Might be a bunch of machinery?
 - ▶ Might just be a person hiding inside?

Procedural Abstraction in Code

- ▶ We can use a function as long as we know **what** it does, without having to know **how** it works.

```
#include <iostream>
#include <vector>
using namespace std;

double mean(vector<double> v) { // <--- interface
    // implementation not shown. a bunch of fancy math
}

int main() {
    vector<double> v;
    // put data in v

    double m = mean(v); // only care about the interface here
    cout << m;
}
```

Project 1 File Structure

Let's look at the project 1 structure as an example of how programs are split into **modules** based on **abstractions...**

We'll also see how **interfaces** are generally separated from their **implementations.**

Project 1 File Structure

- What's wrong here?

main.cpp

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<double> v;
    // put data in v

    double m = mean(v);
    cout << m;
}

double mean(vector<double> v) {
    // implementation not shown
}
```

Compile error here:
mean() has not yet
been declared.

But there's still a design problem...
everything is in the same file!

9

Project 1 File Structure

- Solution: Use a function prototype.

main.cpp

```
#include <iostream>
#include <vector>
using namespace std;

double mean(vector<double> v);

int main() {
    vector<double> v;
    // put data in v

    double m = mean(v);
    cout << m;
}

double mean(vector<double> v) {
    // implementation not shown
}
```

The function prototype acts as a declaration. It specifies the interface of `mean()`, but doesn't fully define it.

A declaration is sufficient at this point.

The definition of the function comes later and gives the implementation of the function.

Project 1 File Structure

- Modularity goes hand in hand with abstraction.
- Let's split our P1 code into multiple files.

main.cpp

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<double> v;
    // put data in v

    double m = mean(v);
    cout << m;
}
```

stats.cpp

```
#include <vector>
using namespace std;

double mean(vector<double> v) {
    // implementation not shown
}
```

A similar problem: main.cpp and stats.cpp are compiled individually, and mean() isn't declared in main.cpp.

Project 1 File Structure

- Solution: Add a header (.h) file for each module (except the driver).

The header contains all relevant declarations.

stats.hpp

```
#include <vector>

double mean(std::vector<double> v);
```

main.cpp

```
#include <iostream>
#include <vector>
#include "stats.hpp"
using namespace std;

int main() {
    vector<double> v;
    // put data in v

    double m = mean(v);
    cout << m;
}
```

stats.cpp

```
#include <vector>
#include "stats.hpp"
using namespace std;

double mean(vector<double> v) {
    // implementation not shown
}
```

Definitions go in the .cpp file.

Give both files to g++.

```
g++ main.cpp stats.cpp -o main.exe
```

Compile Error

The header contains all relevant declarations.

stats.hpp

```
#include <vector>

double mean(std::vector<double> v);
```

main.cpp

```
#include <iostream>
#include <vector>
#include "stats.hpp"
using namespace std;

int main() {
    vector<double> v;
    // put data in v

    double m = mean(v);
    cout << m;
}
```

stats.cpp

```
#include <vector>
#include "stats.hpp"
using namespace std;

double mean(vector<double> v) {
    // implementation not shown
}
```

Definitions go in the .cpp file.

```
main.cpp: In function 'int main()':
main.cpp:9:14: error: 'mean' was not declared in this scope
  9 |     double m = mean(v);
    |                  ^~~~
```

Linking Error

The header contains all relevant declarations.

stats.hpp

```
#include <vector>

double mean(std::vector<double> v);
```

main.cpp

```
#include <iostream>
#include <vector>
#include "stats.hpp"
using namespace std;

int main() {
    vector<double> v;
    // put data in v

    double m = mean(v);
    cout << m;
}
```

stats.cpp

```
#include <vector>
#include "stats.hpp"
using namespace std;

double mean(vector<int> v) {
    // implementation not shown
}
```

Definitions go in the .cpp file.

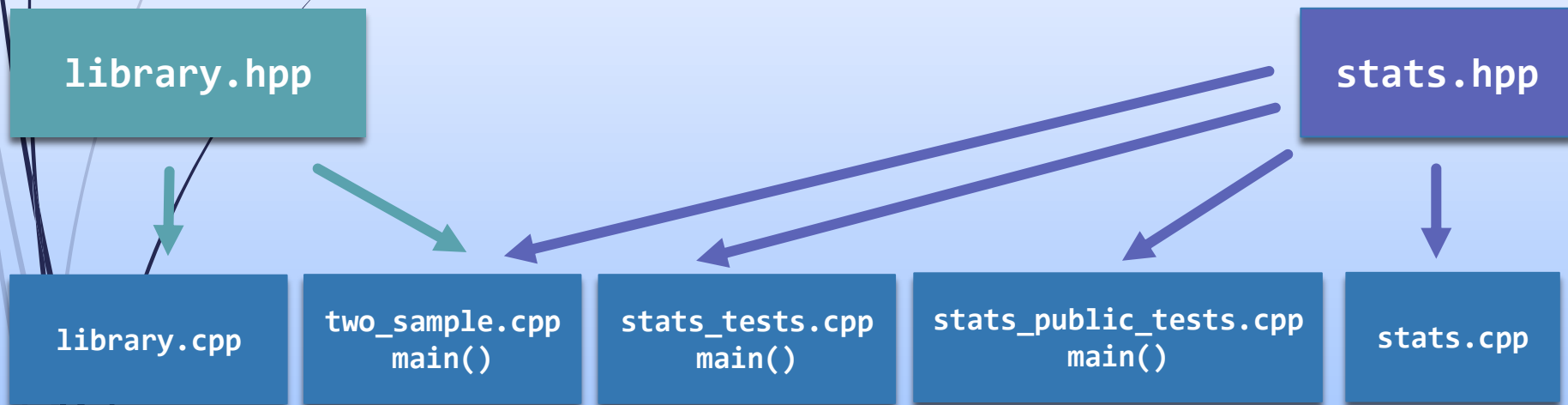
```
/usr/bin/ld: /tmp/cce4axlB.o: in function `main':
main.cpp:(.text+0x43): undefined reference to `mean(std::vector<double, std:
collect2: error: ld returned 1 exit status
```

Project 1 File Structure

➤ Here's the overall structure.

↓ indicates
#include

- Never #include a .cpp file.
- Never put a .hpp file in the g++ command.
- Never compile multiple main() functions together.



```
g++ library.cpp stats.cpp two_sample.cpp -o two_sample.exe
```

```
g++ library.cpp stats.cpp stats_tests.cpp -o stats_tests.exe
```

Makefiles

15

- The make utility can automate compilation/testing.
- A Makefile specifies several **targets** you can run.

```
CXX ?= g++
CXXFLAGS ?= -Wall -Werror -pedantic -g --std=c++17 -Wno-sign-compare -Wno-comment

# Run a regression test
test: stats_public_tests.exe stats_tests.exe two_sample.exe
    ./stats_public_tests.exe
    ./stats_tests.exe
    ./two_sample.exe cats.csv food 1 2 weight > cats.out
    diff cats.out cats.out.correct

# Compile the stats_public_test executable
stats_public_tests.exe: stats_public_tests.cpp stats.cpp
    $(CXX) $(CXXFLAGS) stats_public_tests.cpp stats.cpp -o stats_public_tests.exe
...

# Compile the two_sample executable
two_sample.exe: two_sample.cpp stats.cpp
    $(CXX) $(CXXFLAGS) two_sample.cpp stats.cpp -o two_sample.exe

# Remove automatically generated files
clean :
    rm -rvf *.exe *~ *.out *.dSYM *.stackdump
```

Compile/run ALL public tests:
make test

Compile/run stats.cpp public tests:
make stats_public_tests.exe
./stats_public_tests.exe

Remove all .exe compiled files:
make clean

Specification Comments (RMEs)

- A comment that specifies the **interface** for a function.
- Requires
- Modifies
- Effects

EFFECTS and MODIFIES

- EFFECTS specifies what the function actually does.
 - What is the meaning of the **return value**?
 - Are there any **side effects**? (e.g modifying a data structure)
- MODIFIES indicates which things are potentially changed as a result of side effects.
 - e.g. A reference parameter, cout, global variables, etc.

```
// MODIFIES: v  
// EFFECTS:  sorts v in ascending order  
void sort(std::vector<double> &v);
```

Example: sort has the effect of sorting the input vector v.
Of course, this may modify v.

REQUIRES (i.e. “ASSUMES”)

- Prerequisites for the function to make sense.
- Behavior in cases that break the REQUIRES clause is **undefined** by the **interface**.
- The function **implementation** only needs to cover cases allowed by the **interface's** REQUIRES clause.

```
// REQUIRES: v is not empty  
// EFFECTS: returns median of the numbers in v  
double median(std::vector<double> v);
```

Example: `median` doesn't make any sense for empty `v`.

- A function with no requirements is called **complete**.
- A function with requirements is called **partial**.

REQUIRES

➡ Program: `// REQUIRES: v is not empty
double median(std::vector<double> v);`

➡ Me: `std::vector<double> clearlyEmptyVec;
cout << median(clearlyEmptyVec) << endl;`

➡ Program: *crashes*

➡ Me:



Checking the REQUIRES Clause?

```
// REQUIRES: v is not empty
// EFFECTS: returns median of the numbers in v
double median(std::vector<double> v) {
    if (v.empty()) {
        // try to salvage the situation
    }
}
```

Don't do this.

```
// REQUIRES: v is not empty
// EFFECTS: returns median of the numbers in v
double median(std::vector<double> v) {
    assert(!v.empty()); // sound the alarms!
}
```

Do this.

assert([EXPRESSION])

- ▶ assert() is a programmer's friend for debugging
- ▶ Does nothing if EXPRESSION is true
- ▶ Exits and prints an error message if EXPRESSION is false

```
#include <cassert>
int main() {
    int x = 3;
    int y = 4;
    assert(x < y); // ok, does nothing
    assert(x > y); // crash with debug message
}
```

```
$ ./test
Assertion failed: (false), function main, file
test.cpp, line 6.
```

Motivation for Testing

- Good testing yields correctly working software
 - It's easier to catch bugs than to never make mistakes!
- A thorough set of tests makes development easier
 - Re-run tests after all changes and make sure you didn't break anything!
- Testing is not the same as debugging
 - **Testing:** Discovering that something is broken.
 - **Debugging:** Fixing something once you know it's broken.

Types of Testing

► Unit testing

- One piece at a time (e.g., a function)
- Find and fix bugs early! This saves you time!
 - Test smaller, less complex, easier to understand units.
 - You just wrote the code – so it's easier to debug.

► System testing

- Entire project (code base)
- Do this *after* unit testing

► Regression testing

- Automatically run all unit and system tests after a code change

Kinds of Test Cases

Consider test cases for the mode function from project 1...

```
// REQUIRES: v is not empty
// EFFECTS: Returns the mode of the numbers in v.
//          http://en.Wikipedia.org/wiki/Mode_(statistics)
double mode(std::vector<double> v);
```

Don't
write
these.

Type
Prohibited

```
assert(mode("cat") == "sleeping");
```

REQUIRES
Prohibited

```
vector<double> empty;
assert(mode(empty) == 0);
```

Simple

```
assert(mode({1, 2, 3, 2}) == 2);
```

(Edge)
Special

```
assert(mode({3}) == 3); // single element
assert(mode({1, 2, 1, 2}) == 1); // a tie
```

Generally
not used in
280.

Stress

Take the mode of a really, really big vector.
Used for performance-critical applications like web
servers to test how well they handle an intense load.

Example – Unit Tests

- Let's take a look at some unit tests for project 1.

```
void test_mean_basic() {  
    std::vector<double> data = {1, 2, 3};  
    double expected = 2;  
    double actual = mean(data);  
    assert(actual == expected);  
    ...  
}
```

If this fails, we need to debug
the implementation of mean.

```
int main() {  
    test_mean_basic();  
    test_mean_edge();  
    test_median_basic();  
    ...  
}
```

Debugging mean

- ▶ The essential nature of debugging is to figure out **precisely** where your program goes wrong.
- ▶ We can narrow down where the problem is by observing the state of the program at key points.

```
double mean(vector<double> v) {  
    double s = sum(v);  
    cout << "sum: " << s << endl;  
  
    double c = count(v);  
    cout << "count: " << c << endl;  
  
    return s / c;  
}
```

Think of debugging as hypothesis testing. For example, this line tests the hypothesis "something is wrong with the sum function".

- ▶ Using print statements can be clunky. Throughout the course, you'll learn to use a **visual debugger**.

We'll start again in five minutes.



Participation

bit.ly/469o3DR

